

Python time module practice

```
import time

# =====
# TASK 1 – Get the current Unix timestamp
# =====
# Task:
# Get the current time as a Unix timestamp.
#
# Solution:

current_timestamp = time.time()

print("Current Unix timestamp:", current_timestamp)

# =====
# TASK 2 – Pause program execution
# =====
# Task:
# Print "Starting...", wait 2 seconds, then print "Finished!".
#
# Solution:

print("Starting...")

time.sleep(2)

print("Finished!")

# =====
# TASK 3 – Measure execution time
# =====
# Task:
# Measure how long a piece of code takes to execute.
#
# Solution:

start = time.time()

# Simulate some work
time.sleep(2)

end = time.time()

execution_time = end - start

print("Execution time:", execution_time, "seconds")

# =====
# TASK 4 – Get the current local time
# =====
# Task:
# Get the current local time as a structured time object.
#
# Solution:

current_time = time.localtime()

print("Current local time:", current_time)
```

```

# =====
# TASK 5 - Access individual time components
# =====
# Task:
# Extract:
# - year
# - month
# - day
# - hour
# - minute
# - second
#
# Solution:

current_time = time.localtime()

print("Year:", current_time.tm_year)
print("Month:", current_time.tm_mon)
print("Day:", current_time.tm_mday)
print("Hour:", current_time.tm_hour)
print("Minute:", current_time.tm_min)
print("Second:", current_time.tm_sec)

# =====
# TASK 6 - Get UTC time
# =====
# Task:
# Get the current UTC time.
#
# Solution:

utc_time = time.gmtime()

print("UTC time:", utc_time)

# =====
# TASK 7 - Format time as a string
# =====
# Task:
# Convert the current local time into:
#
# YYYY-MM-DD HH:MM:SS
#
# Example:
# 2026-09-07 07:21:30
#
# Solution:

current_time = time.localtime()

formatted_time = time.strftime(
    "%Y-%m-%d %H:%M:%S",
    current_time
)

print("Formatted time:", formatted_time)

# =====
# TASK 8 - Format only the date
# =====
# Task:
# Create a string containing:
#
# YYYY-MM-DD
#
# Solution:

```

```

current_time = time.localtime()

date_string = time.strftime(
    "%Y-%m-%d",
    current_time
)

print("Date:", date_string)

# =====
# TASK 9 – Format only the time
# =====
# Task:
# Create a string containing:
#
# HH:MM:SS
#
# Solution:

current_time = time.localtime()

time_string = time.strftime(
    "%H:%M:%S",
    current_time
)

print("Time:", time_string)

# =====
# TASK 10 – Create a custom timestamp
# =====
# Task:
# Create this format:
#
# YYYY/MM/DD-HH:MM:SS
#
# Solution:

current_time = time.localtime()

timestamp = time.strftime(
    "%Y/%m/%d-%H:%M:%S",
    current_time
)

print("Timestamp:", timestamp)

# =====
# TASK 11 – Convert a time string into a time object
# =====
# Task:
# Convert:
#
# "2026-09-07 07:30:00"
#
# into a structured time object.
#
# Solution:

time_string = "2026-09-07 07:30:00"

parsed_time = time.strptime(
    time_string,
    "%Y-%m-%d %H:%M:%S"
)

```

```

print("Parsed time:", parsed_time)

# =====
# TASK 12 – Extract information from parsed time
# =====
# Task:
# Parse a timestamp and extract the year, month, and day.
#
# Solution:

timestamp = "2026-09-07 07:30:00"

parsed_time = time.strptime(
    timestamp,
    "%Y-%m-%d %H:%M:%S"
)

print("Year:", parsed_time.tm_year)
print("Month:", parsed_time.tm_mon)
print("Day:", parsed_time.tm_mday)

# =====
# TASK 13 – Convert structured time to Unix timestamp
# =====
# Task:
# Convert a structured local time object into a Unix timestamp.
#
# Solution:

time_data = time.localtime()

timestamp = time.mktime(time_data)

print("Unix timestamp:", timestamp)

# =====
# TASK 14 – Create a simple delay between operations
# =====
# Task:
# Simulate a data pipeline that performs three operations.
# Wait one second between each operation.
#
# Solution:

print("Step 1: Reading data")

time.sleep(1)

print("Step 2: Transforming data")

time.sleep(1)

print("Step 3: Writing data")

time.sleep(1)

print("Pipeline completed")

# =====
# TASK 15 – Build a simple retry mechanism
# =====
# Task:
# Simulate a system that tries to perform an operation.
# If it fails, wait 2 seconds before retrying.
#

```

```

# Solution:
for attempt in range(1, 4):
    print("Attempt:", attempt)

    # Simulate failure
    success = False

    if success:
        print("Operation successful!")
        break

    print("Operation failed.")

    if attempt < 3:
        print("Waiting before retry...")
        time.sleep(2)

else:
    print("All attempts failed.")

# =====
# TASK 16 – Create a simple scheduler
# =====
# Task:
# Run a task every 3 seconds.
#
# Stop after 3 executions.
#
# Solution:
for i in range(3):
    print("Running scheduled task:", i + 1)

    # Simulate task
    print("Processing data...")

    time.sleep(3)

# =====
# TASK 17 – Measure a real operation
# =====
# Task:
# Measure how long a loop takes.
#
# Solution:
start = time.time()

total = 0

for number in range(1, 1_000_001):
    total += number

end = time.time()

print("Total:", total)
print("Execution time:", end - start, "seconds")

# =====
# TASK 18 – Use monotonic() for timing
# =====
# Task:
# Measure elapsed time using a clock designed for measuring

```

```

# durations.
#
# Solution:

start = time.monotonic()

time.sleep(2)

end = time.monotonic()

elapsed = end - start

print("Elapsed time:", elapsed, "seconds")

# =====
# TASK 19 – Compare time.time() and time.monotonic()
# =====
# Task:
# Understand that both can measure elapsed time, but
# monotonic() is safer for duration measurements.
#
# Solution:

start = time.monotonic()

time.sleep(1)

end = time.monotonic()

print("Duration:", end - start)

# =====
# TASK 20 – Data Engineering simulation
# =====
# Task:
# Simulate a streaming pipeline that:
#
# 1. Generates data
# 2. Processes data
# 3. Writes data
#
# Add timestamps to each operation.
#
# Solution:

def get_timestamp():
    return time.strftime(
        "%Y-%m-%d %H:%M:%S",
        time.localtime()
    )

print(get_timestamp(), "- Generating data")

time.sleep(1)

print(get_timestamp(), "- Processing data")

time.sleep(1)

print(get_timestamp(), "- Writing data")

time.sleep(1)

print(get_timestamp(), "- Pipeline completed")

```

```
# =====
# IMPORTANT TIME FUNCTIONS
# =====
#
# time.time()
#     -> Current Unix timestamp
#
# time.sleep(seconds)
#     -> Pause execution
#
# time.localtime()
#     -> Current local time
#
# time.gmtime()
#     -> Current UTC time
#
# time.strftime()
#     -> Time object -> formatted string
#
# time.strptime()
#     -> String -> structured time object
#
# time.mktime()
#     -> Local time object -> Unix timestamp
#
# time.monotonic()
#     -> Monotonic clock for measuring durations
#
# =====
```